CUXR Onboarding Project

What you will build

- A live webcam application that detects faces and draws bounding boxes.
- A face embedding pipeline that converts an aligned face into a numeric vector.
- A recognizer that compares a new vector against enrolled people using cosine similarity.
- A persistent memory layer that stores first seen, last seen, and encounter history.
- A debouncer / presence tracker that prevents one continuous interaction from being counted hundreds of times.
- A version of the app deployed on our RayNeo smart glasses.

Dataset use

All face images used in this project come from the publicly available Labeled Faces in the Wild (LFW) dataset. Download the dataset programmatically with scikit-learn, keep the raw files in a local `data/lfw/` directory, and use the dataset's person labels as the identity metadata. You will be guided through this later in the project

1 System architecture

The most important architectural idea is separation of concerns: camera/device I/O should be independent from face-recognition logic. That lets a laptop webcam and RayNeo X3 Pro feed the same recognition and memory modules.

Module	Input	Output	Beginner mental model
Camera adapter	Video device	Frames	A stream of images
Face detector	Frame	Boxes + landmarks	Where are the faces?
Align / crop	Frame + landmarks	Normalized face	Put the face in a standard pose
Embedding model	Aligned face	Vector	Turn appearance into numbers
Matcher	Query vector + known vectors	Best score + ID	Who is this most similar to?
Decision threshold	Best score	Known / Unknown	Is the match good enough?
Memory / debounce	Identity + time	Encounter state	Have we already counted this interaction?

Module	Input	Output	Beginner mental model
UI / AR overlay	Identity + memory	Display	Show useful context to the wearer

Architecture rule. Do not put camera code, model code, database code, and UI code in one giant `main.py`. Each module should be testable on its own.

Frame-by-frame recognition flow

For one camera frame: (1) detect a face, returning bounding box and landmarks; (2) align and crop it to a normalized face image; (3) encode it as a fixed-length embedding; (4) compare it with known embeddings; (5) apply a threshold to the best score; and (6) update Known/Unknown status and memory. A cosine similarity score is not a probability. Calibrate the threshold on your own data.

2 ML concepts you actually need

2.1 Images are arrays

A color image is a grid of pixel values. OpenCV typically stores a color frame as a NumPy array shaped like `(height, width, channels)`. Video is simply a sequence of frames processed repeatedly.

```
import cv2

image = (lfw.images[0] * 255).astype('uint8')
image = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
print(image.shape)        # example: (125, 94, 3)
print(image[100, 100])    # one BGR pixel
```

2.2 Detection is not recognition

Task	Question	Typical output
Face detection	Is there a face, and where?	Bounding box + landmarks
Face recognition	Does this face match a known identity?	Identity + similarity score

2.3 Inference

Inference means running an already-trained model. For onboarding, you are not training the detector or face encoder. You are using pretrained models and learning how to integrate, evaluate, and reason about them.

2.4 Embeddings

An embedding is a fixed-length vector that represents a face in a learned feature space. Photos of the same person should generally produce vectors that are closer than photos of different people, but the separation is not perfect.

```
# Toy example only - real face embeddings contain many more values.
alice_photo_1 = [ 0.21, -0.43, 0.91, 0.17]
alice_photo_2 = [ 0.23, -0.40, 0.89, 0.19]
bob_photo_1   = [-0.61, 0.72, -0.12, 0.53]
```

2.5 Cosine similarity

Cosine similarity compares the direction of two vectors. The normalized dot product is

$$\text{similarity}(a, b) = \frac{a \cdot b}{\|a\| \, \|b\|}.$$

```
import numpy as np

def cosine_similarity(a, b):
    a = np.asarray(a, dtype=np.float32)
    b = np.asarray(b, dtype=np.float32)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

This is not probability. A cosine similarity score is not automatically a calibrated probability. Display it as a match/similarity score unless the system has been calibrated to produce probabilities.

2.6 Thresholds and errors

Term	Meaning	Example
Threshold	Minimum similarity required to accept a match	score $\geq$ threshold $\rightarrow$ known person
False positive	Different person incorrectly accepted as a known identity	Bob recognized as Alice
False negative	Correct person rejected / labeled unknown	Alice recognized as Unknown

OpenCV publishes example SFace thresholds for benchmark datasets, but those values are references and may not apply in all settings. You will need to find an adequate threshold.

3 Environment setup

3.1 Prerequisites

- Python 3.10+
- Git

- VS Code
- The publicly available LFW face dataset

3.2 Create the project

```
mkdir facial-recognition-onboarding
cd facial-recognition-onboarding

python -m venv .venv

# macOS / Linux
source .venv/bin/activate

# Windows PowerShell
# .venv\Scripts\Activate.ps1

python -m pip install --upgrade pip
pip install opencv-python numpy scikit-learn matplotlib
```

3.3 Get the reference models

For the reference implementation, use OpenCV Zoo models: YuNet for face detection and SFace for face recognition. Download the ONNX model files from the OpenCV Zoo repository and place them in `models/`.

```
models/
|-- face_detection_yunet_2023mar.onnx
‘-- face_recognition_sface_2021dec.onnx
```

Version note. Model filenames in OpenCV Zoo can change over time. If the exact filename above is replaced, use the current YuNet detector and SFace recognizer listed in OpenCV Zoo.

3.4 Recommended workflow

The six onboarding checkpoints are: 0 Python + JSON; 1 images + webcam; 2 face detection; 3 embeddings; 4 recognition DB; 5 memory + RayNeo map.

1. Finish one checkpoint before moving to the next.
2. Commit after every working checkpoint.
3. Write down what changed and what failed.

4 Checkpoint 0 – Python and state

Before getting to the ML parts, learn how the application will represent people and update state.

Task

```
people = {
    'Alice': {'times_seen': 3, 'last_seen': '2026-09-22T14:30:00'},
    'Bob':   {'times_seen': 1, 'last_seen': '2026-09-22T13:15:00'},
}

def record_person(name, timestamp):
    if name not in people:
        people[name] = {
            'times_seen': 0,
            'first_seen': timestamp,
            'last_seen': timestamp,
        }
    people[name]['times_seen'] += 1
    people[name]['last_seen'] = timestamp
```

Then save and reload the dictionary with JSON. The checkpoint is complete when restarting the program does not erase the stored state.

What you should be able to explain

- Why a dictionary is useful for person records
- Why timestamps should use a consistent format
- The difference between in-memory state and persistent storage

5 Checkpoint 1 – dataset images and webcam interface

All still images for this checkpoint come from LFW. The webcam code below is included to teach the eventual device-input interface; do not save webcam frames or use them as enrollment/evaluation images.

Download the public dataset

```
from sklearn.datasets import fetch_lfw_people

lfw = fetch_lfw_people(
    data_home='data',
    min_faces_per_person=20,
    resize=0.5,
    color=True,
)
print(lfw.images.shape)       # (num_images, height, width, 3)
print(lfw.target_names[:5])   # identity labels
```

Use a fixed random seed to split each identity into enrollment and evaluation images. Keep the split person-disjoint where a separate unknown-identity test is needed, and never use the same image in both the gallery and evaluation set.

Read an image

```
import cv2

image = cv2.imread('data/lfw/<downloaded-image>.jpg')
print(image.shape)

small = cv2.resize(image, (640, 360))
gray = cv2.cvtColor(small, cv2.COLOR_BGR2GRAY)
cv2.imwrite('gray.jpg', gray)
```

Read live video

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    ok, frame = cap.read()
    if not ok:
        break

    cv2.imshow('Camera', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

Checkpoint exercise

- Display the frame width and height.
- Add an FPS counter.
- Save one frame when the user presses S.
- Explain why processing every frame can become expensive on wearable hardware.

6 Checkpoint 2 – face detection

Now add the first pretrained ML model. The detector locates faces and returns landmarks. The landmarks are useful because the recognizer can align the face before extracting an embedding.

```
import cv2

DETECTOR_MODEL = 'models/face_detection_yunet_2023mar.onnx'

detector = cv2.FaceDetectorYN.create(
    DETECTOR_MODEL,
```

```
    '',
    (320, 320),
    0.9,    # score threshold
    0.3,    # NMS threshold
    5000,   # top_k
)

# inside the webcam loop:
h, w = frame.shape[:2]
detector.setInputSize((w, h))
_, faces = detector.detect(frame)

if faces is not None:
    for face in faces:
        x, y, fw, fh = face[:4].astype(int)
        cv2.rectangle(frame, (x, y), (x + fw, y + fh), (0, 255, 0), 2)
```

Debugging question. If detection works in bright frontal lighting but fails at a sharp angle or in darkness, is that a bug in your loop or a limitation of the model/input? How might this be addressed?

Checkpoint acceptance criteria

- At least one face is detected from the webcam.
- Bounding boxes remain reasonably stable while the person moves.
- The code handles the no-face case without crashing.
- You can explain what the detector output represents.

7 Checkpoint 3 – embeddings and similarity

This is the core ML idea in the project. The recognizer aligns the detected face, extracts a feature vector, and compares vectors.

```
RECOGNIZER_MODEL = 'models/face_recognition_sface_2021dec.onnx'
recognizer = cv2.FaceRecognizerSF.create(RECOGNIZER_MODEL, '')

def get_embedding(frame, face_row):
    aligned = recognizer.alignCrop(frame, face_row)
    feature = recognizer.feature(aligned)
    return feature.flatten().astype('float32')
```

Similarity experiment

1. Select 3–5 LFW images for each identity, preserving the dataset labels.
2. Select images from at least one different LFW identity.
3. Generate an embedding for every dataset image.
4. Compute all same-person and different-person cosine scores.

5. Put the scores into a table and inspect where the two groups overlap.

Image A	Image B	Same person?	Cosine score
Alice 1	Alice 2	Yes	0.XX
Alice 1	Alice 3	Yes	0.XX
Alice 1	Bob 1	No	0.XX
Alice 2	Bob 2	No	0.XX

The actual values are the point of the exercise. Do not copy a made-up threshold. Plot or inspect your distributions, then choose a threshold and explain the tradeoff between false accepts and false rejects.

8 Checkpoint 4 – enrollment and recognition

8.1 Enrollment

Enrollment means intentionally adding a known LFW identity. Select multiple reference images from that identity's LFW images, generate embeddings, and save them under a stable person ID. Keep the dataset identity label separate from the internal ID so the data layer remains easy to change without rewriting embeddings.

```
data/
|-- embeddings.npz
`-- people.json

# people.json (illustrative person-ID mapping)
{
  'person_001': {'display_name': 'Alice'},
  'person_002': {'display_name': 'Bob'}
}
```

Better than one photo. Store several reference embeddings per person or an averaged/representative embedding. One enrollment photo is fragile under pose, lighting, glasses, hairstyle, and camera changes.

8.2 Match a query

```
THRESHOLD = 0.36   # example starting point for OpenCV SFace; calibrate locally

def identify(query_embedding, gallery):
    best_id = None
    best_score = -1.0

    for person_id, reference_embeddings in gallery.items():
        for ref in reference_embeddings:
            score = cosine_similarity(query_embedding, ref)
            if score > best_score:
                best_score = score
                best_id = person_id
```

```
    if best_score >= THRESHOLD:
        return best_id, best_score
    return None, best_score
```

Threshold warning. 0.36 is included only as an OpenCV SFace benchmark reference. It is not a threshold that is guaranteed to work. You will likely need to recalibrate this.

8.3 Unknown handling

- Do not force every face into the nearest known identity.
- Return Unknown when the best score is below threshold.

9 Checkpoint 5 – persistent memory and debouncing

A camera may process 30 frames per second. Without state tracking, a person who stands in front of the camera for 10 seconds could appear as hundreds of separate "encounters." The memory layer needs to reason about sessions, not frames.

9.1 Minimal memory record

```
{
    'person_001': {
      'first_seen': '2026-09-22T15:20:13',
      'last_seen': '2026-09-22T15:43:28',
      'encounter_count': 6,
      'active': true,
      'last_detection': '2026-09-22T15:43:28'
    }
}
```

9.2 Beginner debouncer

```
ABSENCE_GAP_SECONDS = 10

# Idea:
# 1. While a person is continuously visible, update last_detection only.
# 2. Mark the person inactive after they have not been seen for ABSENCE_GAP_SECONDS.
# 3. The next time an inactive person becomes visible, increment encounter_count.
```

This is better than incrementing a counter every N seconds because a long uninterrupted conversation should remain one encounter. A future production system could use face tracking / track IDs to make session boundaries more robust, but that's not needed for now.

9.3 Display example

```
Alice
Match score: 0.47
Encounters: 4
Last seen: 2 minutes ago
```

UI principle. Prefer "match score" over "97% confident." Similarity values only become probabilities after explicit calibration.

10 RayNeo X3 Pro integration

After the recognition pipeline works on a laptop, adapt it to the RayNeo X3 Pro. RayNeo supports application development through Android and Unity ARDK, so the glasses version should be implemented as a small application that connects the camera, recognition pipeline, and AR display.

10.1 Glasses version

The glasses application should:

1. Read live frames from the RayNeo camera.
2. Detect faces using YuNet or another supported face detector.
3. Align detected faces and generate embeddings using SFace.
4. Allow the user to enroll a person by collecting several images of their face.
5. Store the enrolled embeddings under that person's ID.
6. Compare new face embeddings against the enrolled gallery.
7. Display the recognized person's name, match score, or "Unknown" in the glasses display.
8. Use the debouncing logic from Checkpoint 5 so that one continuous interaction counts as one encounter.

10.2 Laptop-to-glasses mapping

Laptop component	Glasses implementation
Camera input	RayNeo camera frames
Face detection	YuNet running in the glasses application
Face alignment	Same alignment procedure used on the laptop
Face embeddings	SFace inference on each enrolled or detected face
Recognition	Cosine similarity and the calibrated threshold
Memory	Encounter and last-seen state from Checkpoint 5
Display	Name, match score, and bounding box in the AR view

10.3 Implementation sequence

1. Run the camera and display on the glasses.
2. Add face detection and draw bounding boxes.
3. Add face alignment and SFace embeddings.
4. Add an enrollment flow that collects several images of each person.
5. Add recognition and display the predicted identity.
6. Add encounter memory and test the system while people move in and out of view.

The laptop and glasses versions should share the detector, alignment, embedding, matching, and memory logic as much as possible. Only the camera-input and display code should need to change substantially. RayNeo's developer resources are available at `https://open.rayneo.com/`.

11 Testing and evaluation

11.1 Functional tests

Test	Expected behavior
No face in frame	No crash; no identity emitted
One enrolled face	Correct person above threshold
Unenrolled face	Unknown
Two faces	Each face processed independently
Person remains in view	One continuous encounter
Person leaves and returns after gap	New encounter
Restart application	Memory survives restart

11.2 Evaluation set

- Use held-out LFW images from the same identities for evaluation.
- Use the pose, lighting, and image-quality variation already present in LFW.
- Keep a portion of dataset images out of enrollment so evaluation is not identical to the gallery.
- Record false accepts and false rejects at several thresholds.

12 Repository structure and review checklist

```
facial-recognition-onboarding/
|-- README.md
|-- requirements.txt
|-- models/
|   |-- face_detection_yunet_*.onnx
|   `-- face_recognition_sface_*.onnx
|-- data/
```

```
|   |-- embeddings.npz
|   `-- people.json
|-- experiments/
|   `-- similarity_scores.csv
|-- src/
|   |-- camera.py
|   |-- detector.py
|   |-- embeddings.py
|   |-- recognition.py
|   |-- memory.py
|   `-- main.py
`-- tests/
    |-- test_similarity.py
    |-- test_recognition.py
    `-- test_memory.py
```

Code review checklist

- No giant monolithic `main.py`.
- No hard-coded personal paths.
- No silent auto-enrollment.
- Handles no-face and multi-face cases.
- Threshold is a configuration value and documented.
- Similarity score is not presented as a probability.
- Memory is persisted and has a deletion path.
- At least one experiment documents same-person vs different-person scores.
- README explains how to set up models and run the demo.

Knowledge review

1. What is the difference between detection and recognition?
2. What does inference mean?
3. What is an embedding?
4. Why are raw pixels not a good identity comparison?
5. What does cosine similarity measure?
6. Why do we need a threshold?
7. What are false accepts and false rejects?
8. Why use multiple enrollment images?
9. Why is a match score not a probability?
10. Why do we need debouncing / encounter sessions?
11. Which parts of the laptop system should stay unchanged when moving to RayNeo?